CacheFlow: Caching Generated Reasoning, Not Just the Prompt, for Persistent-Memory Coding Agents

Agastya Choudhary
agastya.r.choudhary@gmail.com

July 8, 2026

Abstract

Coding agents pay for the same computation twice. On *self-hosted* models the prompt prefix—a system prompt plus tens of thousands of tokens of codebase context—is re-evaluated on every invocation even though its attention (KV) state is deterministic and reusable. On *cloud-hosted* models the situation is worse: provider prompt caching amortizes the stable prefix, but *extended-thinking tokens*—the reasoning a model generates before answering—are produced fresh on every call and cannot be cached at all, because they are generated output, not cached input. In multi-agent and multi-step workflows, independent agents re-reason about the same problem and re-read the same files, paying full reasoning cost each time. We present CacheFlow, a persistent-memory layer that addresses both costs with mechanisms matched to what each deployment actually exposes. For self-hosted `llama.cpp` models, CacheFlow serializes and restores per-sequence KV state, turning a multi-second cold prefill into a sub-second warm restore. For cloud models, where no internal state is retrievable, it instead caches the model's *outputs*: a content-addressed *thinking-block store* with confidence-gated semantic reuse. Across eight real-world Python corpora we measure a cold prime replaced by a ∼300 ms warm restore, cumulatively ∼246 s of wall-clock prefill and ∼1,500 TFLOP of compute avoided on the self-hosted path, and exact, API-reported reductions of up to 100% of a turn's reasoning tokens on the cloud path, with all savings computed from real token counts rather than estimated from text length.

1 Introduction

Large-language-model coding agents are increasingly deployed as *long-lived, multi-step, multi-agent* systems: a planner delegates to an implementer, a reviewer inspects the diff, a tester exercises the change, and each of these may run dozens of tool-calling iterations. Every one of these invocations begins by presenting the model with a large, mostly-stable context (a system prompt plus a substantial slice of the codebase) and then asks the model to reason toward an action.

Two distinct, recurring costs arise from this pattern, and—critically—they have *different* solutions depending on whether the model is self-hosted or cloud-hosted:

1. **Prefix re-evaluation (self-hosted).** A self-hosted model exposes its own attention/KV state. Yet most serving stacks discard that state between calls and re-run the full prefill over the stable prefix, spending seconds of wall-clock time and hundreds of TFLOPs re-deriving a state that is deterministic given the same tokens and model.
2. **Reasoning regeneration (cloud-hosted).** A cloud API never exposes internal state; there is no `llama_state_seq_get_data` equivalent to call. Provider-side prompt caching [5] amortizes the *input* prefix, but *extended-thinking* tokens are *generated output*. They are recomputed on every call and, once a provider's short cache TTL lapses, cannot be shared even between two agents solving the identical problem minutes apart.

The central observation of this paper is that these are two faces of the same waste—*recomputing a deterministic function of the same inputs*—but that they must be attacked at different layers because cloud and self-hosted deployments expose different surfaces. Where the model's internals are reachable, cache the *internals* (KV state). Where they are not, cache the model's *outputs* (reasoning traces and derived understanding).

CacheFlow implements both. It began as a KV-cache restoration system for self-hosted `llama.cpp` models (Section 5) and was extended with an output-caching layer for cloud models (Section 6). The two

subsystems share design philosophy—content-hash-based staleness, snapshot/GC lifecycle, exact rather than estimated metrics—but are fully independent in storage and invalidation.

Contributions.

- A persistent per-sequence KV-cache snapshot/restore system for self-hosted models that skips the redundant warm-path save and forces a safe re-prime on codebase or model change (Section 5).
- An output-caching layer for cloud models: a content-addressed, confidence-gated *thinking-block store* (Section 6).
- A discipline of *exact* token accounting: every savings figure is a real, provider-reported token count, never estimated from character length (Section 8).
- An evaluation across eight real-world Python corpora quantifying both paths, including honest reporting of zero-hit cases (Section 9).

2 Background and Motivation

2.1 The Prefill Tax on Self-Hosted Models

Transformer inference proceeds in two phases: a compute-bound *prefill* that populates the KV cache for every prompt token in parallel, and a bandwidth-bound *decode* that emits output tokens one at a time. For a coding agent whose prompt is dominated by a fixed system prompt and codebase slice, prefill is pure overhead after the first call: the KV state for those tokens is identical across invocations of the same model. A multi-thousand-token codebase prefill on an 8B model costs on the order of tens of seconds of wall-clock time and hundreds of TFLOPs in our measurements (Section 9: e.g. a cold prime of ∼20 s on the `requests` corpus), all of it re-derivable from disk in milliseconds if the state is persisted.

2.2 The Reasoning Tax on Cloud Models

Extended thinking lets a model emit an internal reasoning trace before its final answer, materially improving performance on hard tasks. But this trace is *output*: it is billed as output tokens and regenerated on every call. Consider a single agent running a 20-step tool-use loop at ∼5,000 thinking tokens per step: 100,000 reasoning tokens, none of which any provider cache can eliminate. In a multi-agent workflow the same reasoning is paid two or three times as each agent independently re-derives it. And because provider caches expire on the order of minutes, an agent that starts even slightly after another gets no benefit from the first agent's work.

2.3 Why Existing Techniques Are Insufficient

Provider prompt caching [5] and prefix-reuse systems such as PromptCache [2] and PagedAttention [1] target the *input* prefix; they do not touch generated reasoning. Context-compaction and summarization reduce message bloat but do not stop each agent from thinking independently. Retrieval-augmented generation [3] injects relevant documents but does not cache the *conclusions* an agent reaches. CacheFlow's cloud path is complementary to all of these: it caches the reasoning and the derived understanding themselves.

3 Problem Formulation

We now make the two costs of Section 1 precise and derive the condition under which caching is profitable. Table 1 fixes notation.

3.1 Self-Hosted: the Prefill Recurrence

Without persistence, every one of the N calls re-prefills the whole prefix, so total prefill cost is

$$C_{\text{base}} = N\,(P+s)\,c_{\text{pre}}. \tag{1}$$

With snapshot persistence the first call primes (P tokens) and each subsequent call restores the snapshot and prefills only its suffix:

$$C_{\text{cf}} = \underbrace{P\,c_{\text{pre}}}_{\text{one cold prime}} + (N-1)\big(c_{\text{res}} + s\,c_{\text{pre}}\big). \tag{2}$$

Table 1: Notation.

Symbol	Meaning
P	length (tokens) of the stable prefix (system prompt + code)
s	length of the per-call task suffix, $s \ll P$
N	number of invocations sharing the same prefix/problem
c_{pre}	cost to prefill one token (compute or wall-clock)
c_{res}	cost to restore a KV snapshot from disk
T_r	extended-thinking tokens generated on a cold reasoning turn
T_v	tokens spent validating a borderline cached block
$\sigma(\cdot)$	similarity score of a candidate cached artifact
h	content hash; h_w at write time, h_r at read time
p	empirical reuse (cache-hit) rate over the N calls
θ_u, θ_v	use-directly and validate confidence thresholds

Since $c_{\text{res}} \approx 0$ relative to $P\,c_{\text{pre}}$ and $s \ll P$, the per-warm-call saving approaches $P\,c_{\text{pre}}$ and the speedup grows with prefix size:

$$\frac{C_{\text{base}}}{C_{\text{cf}}} \xrightarrow[s,\,c_{\text{res}}\to 0]{} \frac{N(P+s)}{P} \;\approx\; N. \tag{3}$$

The compute form of c_{pre} is not constant in P: a decoder layer's attention contributes a term quadratic in sequence length. For a model of Π parameters, L layers, and hidden width d, the FLOPs avoided by skipping a prefill of n tokens is

$$\mathcal{F}(n) \;=\; \underbrace{2\,\Pi\,n}_{\text{dense (linear)}} \;+\; \underbrace{2\,L\,d\,n^2}_{\text{self-attention (quadratic)}}. \tag{4}$$

We evaluate Equation (4) with Π, L, d read exactly from the loaded GGUF (Section 8), never estimated from a model-name size tag.

3.2 Cloud: Expected Reasoning Cost Under Reuse

On the cloud path there is no c_{res} to exploit; the recurring cost is the reasoning T_r regenerated per call. Let p be the fraction of calls that find a usable cached block. A cold call costs T_r; a validated reuse costs only T_v (the confirmation call, $T_v \ll T_r$); an exact/high-confidence reuse costs 0. Modelling the hit population as a fraction α of exact or high-confidence hits and $1-\alpha$ validated hits, the expected per-call reasoning cost is

$$\mathbb{E}[T] \;=\; (1-p)\,T_r \;+\; p\big[(1-\alpha)\,T_v\big], \tag{5}$$

and the fractional reasoning saving is

$$\eta \;=\; 1 - \frac{\mathbb{E}[T]}{T_r} \;=\; p\Big(1 - (1-\alpha)\tfrac{T_v}{T_r}\Big). \tag{6}$$

Equation (6) makes the design tension explicit: savings rise with hit rate p, but a reused block that is *wrong* corrupts the downstream call, so p must be earned conservatively rather than maximised (Section 6.1).

3.3 The Reuse Decision

Retrieval returns the best candidate and a calibrated confidence $\kappa \in [0,1]$; the action is a thresholded decision

$$a(\kappa) = \begin{cases} \text{USE\_\_DIRECTLY} & \kappa \ge \theta_u, \\ \text{VALIDATE} & \theta_v \le \kappa < \theta_u, \\ \text{RE\_\_THINK} & \kappa < \theta_v. \end{cases} \tag{7}$$

Confidence combines raw similarity with an age penalty; for a block written t days ago,

$$\kappa \;=\; \sigma(\cdot)\cdot e^{-\lambda t}, \qquad \lambda = 0.1, \tag{8}$$

a $\sim$7-day half-life reflecting that older reasoning was computed against a codebase that has since drifted. When the pool holds ≥ 8 blocks we replace the absolute σ with a scale-invariant background z-score

$$z \;=\; \frac{\sigma^\star - \mu_\sigma}{\varsigma_\sigma}, \tag{9}$$

Algorithm 1: Restore-or-Prime session (self-hosted)

Input: agent A, stable context X, active model M

$h_r \leftarrow \textsc{hash}(X)$

if $A.head \neq \varnothing \wedge A.head.h_w = h_r \wedge A.head.model = M$ **then**

  kv $\leftarrow$ RestoreSnapshot(A.head) // warm, $O(c_{\text{res}})$; save skipped

else

  kv $\leftarrow$ Prefill(X) // cold: $O(P\, c_{\text{pre}})$

  snap $\leftarrow$ SerializeSeq(kv)

  WriteSnapshot(snap); A.head $\leftarrow$ snap // write before HEAD update

$y \leftarrow$ Complete(kv, suffix)

RecordMetrics(A); **return** y

where $\sigma^\star$ is the top candidate's similarity and $\mu_\sigma, \varsigma_\sigma$ are the mean and standard deviation of the query's similarity to the rest of the pool—measuring how far the best hit *stands out*, which transfers across domains where the absolute scale of σ does not.

3.4 Staleness as a Hash Predicate

Both subsystems replace explicit invalidation with a write-time/read-time hash comparison. An artifact is served iff

$$\textsc{valid} \iff h_r = h_w, \tag{10}$$

so any edit to the underlying prefix, file region, or model silently changes h_r and suppresses the stale artifact with no separate expiry path.

4 System Overview

CacheFlow is a command-line tool (`cf`) and a set of libraries packaged as a single Python distribution. It comprises two independent subsystems joined by a shared philosophy but no shared state:

Self-hosted KV engine
Serializes and restores per-sequence attention state for local `llama.cpp` models (Section 5).

Cloud output cache
A SQLite-backed pool, `ThinkingStore`, that caches generated reasoning for cloud-hosted agents (Section 6).

Both subsystems detect staleness the same way: a content hash computed at *write* time is compared at *read* time (Equation (10)), so a changed codebase or model silently invalidates the cached artifact with no explicit expiry logic. They store to distinct SQLite databases (`.cacheflow/agents.db`, `thinking.db`) and share no foreign keys: a KV-snapshot restore and a thinking-block hit are unrelated events that can each occur independently (Figure 1).

5 The Self-Hosted KV-Cache Engine

5.1 Core Flow: Restore/Prime → Save → Complete → Record

Each agent maintains a HEAD pointer to at most one KV snapshot. A session proceeds as:

1. **Restore or Prime.** The agent computes `stable_context` (system prompt + codebase text) and its hash. If the agent's HEAD snapshot's `stable_context_hash` still matches, the snapshot is restored (*warm path*). Otherwise the prefix is fed to the model to populate the KV cache (*cold prime*).
2. **Save.** On the prime path *only*, the KV cache is serialized to disk. On the warm path this is skipped: the HEAD snapshot on disk is already byte-identical.
3. **Complete.** The model generates using prefix matching—cached tokens plus the short (∼5–50 token) task suffix.
4. **Record.** The agent's HEAD and token/time metrics are updated in SQLite; savings are computed against the cold-prime baseline via Equations (1) and (2).

Algorithm 1 makes the two safety invariants explicit: the warm branch is taken only when *both* the content hash and the model match (a snapshot is raw KV bytes tied to one tokenizer/vocabulary), and the snapshot is written *before* HEAD is advanced, so a crash leaves A on its previous valid snapshot.

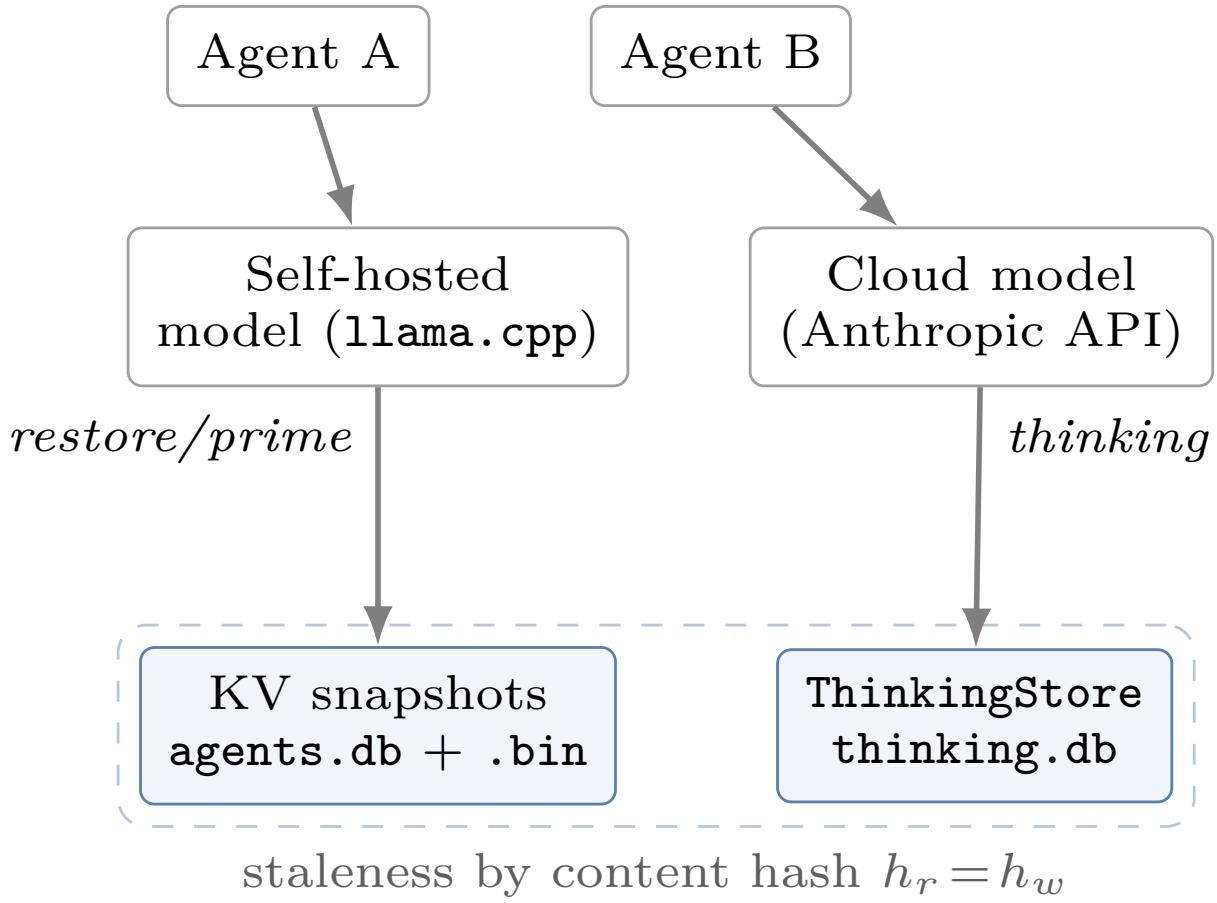

Figure 1: CacheFlow's two independent subsystems. Where model internals are reachable (self-hosted), it caches KV state; where they are not (cloud), it caches generated *reasoning*. No store shares keys with another.

5.2 Per-Sequence Snapshots (Format v4)

Rather than serialize the full `n_ctx` buffer, CacheFlow saves only the live KV for the sequence via `llama_state_seq_get_data`, yielding compact snapshots (∼8 MB of in-memory slot state per idle agent). Because a snapshot is raw KV bytes tied to one model's tokenizer, vocabulary, and hidden dimensions, a `model_changed` guard in `AgentSession` forces a re-prime (never a restore) whenever the active model differs from the one that produced the snapshot.

5.3 Concurrency, Consolidation, and GC

A `SlotPool` manages up to eight concurrent KV slots with a `SlotLease` context manager and LRU eviction that never evicts an actively-running agent. Past 70% of the context window, a background `Compressor` restores HEAD, asks the model for a ≤500-token knowledge summary, and folds it into the next session's stable prefix. A `SnapshotGC` reaps snapshot files unreferenced by any agent's HEAD, plus crash orphans; the write-snapshot-then-update-HEAD ordering guarantees a crash leaves the agent on its previous valid snapshot.

5.4 In-Process Execution

The model runs in-process via `llama-cpp-python` rather than behind an HTTP shim. On macOS an HTTP transport throttles decode by roughly 10×; in-process execution avoids that path. Decode remains bandwidth-bound at ∼5 tokens s^{-1}, so the engine optimizes the compute-bound prefill (`n_batch`=`n_ubatch`=2,048) where caching pays off, and a single global model instance (`get_global_engine`) is shared across all agents.

The prefill batch width is a deliberate choice. `llama.cpp` decodes a prompt in chunks of `n_batch` tokens; the default 512 forces a multi-thousand-token codebase prefix through many sequential chunks. Raising `n_batch=n_ubatch` to 2,048 lets the same prefix populate in a quarter as many passes, and the engine enables FlashAttention so that a stable context spanning several batch chunks is attended to in a single fused kernel with $O(d)$ rather than $O(n)$ intermediate memory per token—a prerequisite for priming real-codebase contexts that exceed one batch width. A one-time `atexit` teardown then frees the inference model and the vocabulary-only tokenizer model (Section 8) in a fixed order, since both hold buffers in the same `ggml` Metal device manager and must be released deterministically rather than at interpreter-shutdown GC order.

5.5 Per-Model Instruction Templating

A KV snapshot is only reusable if every session wraps its turns in the byte-exact token markers the model was trained on; a prefix that drifts by a single control token no longer matches the cached KV. CacheFlow therefore selects an instruction template per model rather than hard-coding one family. A `ChatTemplate` record fixes the system/user wrappers, the assistant open/close markers, and the generation stop token for each of five families (ChatML, Llama 3, Mistral, Gemma, Phi 3). `detect_template` resolves the family by a three-stage cascade, ordered by reliability:

1. **Metadata sniff.** If the GGUF's embedded `tokenizer.chat_template` (exposed as `Llama.metadata`) contains a distinctive marker—`<|start_header_id|>`, `[INST]`, `<start_of_turn>`—the family is read straight from the model file.
2. **Name match.** Otherwise the model name is matched against known family strings.
3. **Default.** Failing both, ChatML is assumed.

Families with no dedicated system role (Mistral, Gemma; `supports_system`=FALSE) fold the system prompt into the first user turn rather than dropping it, and the agentic loop's generation `stop` list uses the family's own `stop_token` so multi-turn assistant boundaries terminate correctly on every model.

5.6 The Agentic Loop

`cf agent` drives an observe–act loop (`run_agentic`) that is deliberately decoupled from `AgentSession`: it touches only the KV-cache-facing primitives an external harness would have (`_restore_or_prime`, `server.completion`, lock acquire/release). The codebase KV stays resident in one slot for the whole loop, so every step prefix-matches the cached prefix and evaluates only the new observation plus the generated action. Each step emits a Thought/Action/Args triple parsed and dispatched against a tool registry (`read_file`, `edit_file`, `write_file`, `grep`, `list_dir`, `syntax_check`, `run_bash`, `finish`), extended with native pool tools (Section 6). Two termination conditions bound the loop beyond `max_steps`: (i) if a completion is byte-identical to the previous one twice consecutively—the signature of deterministic decoding re-emitting the same malformed action against identical error feedback—the loop halts with a `stuck_loop` step; (ii) before each completion, if the prompt's token count plus `max_tokens_per_step` would meet or exceed `ctx_size`, it halts with a `context_limit` step. Both return the partial trajectory rather than raising, so a stalled model degrades to a truncated result instead of a crash.

Sliding Window: Reusing a Fixed Prefix Across Many Steps

The agentic loop exploits a property of multi-step reasoning: the stable codebase context is deterministic across all steps, so its KV state is restored *once* and reused for every subsequent step. As the loop progresses, the conversation history (observations, tool outputs, actions) accumulates in an implicit *sliding window* where the cached prefix remains fixed while the prompt grows forward against a context ceiling (Figure 2).

Let P be the prefix length (system prompt + codebase), H_i the cumulative conversation history through step i, and Λ the total context window (`ctx_size`). The prompt assembled at step i is:

$$\text{prompt}_i \;=\; P \;+\; H_i \;+\; s_i, \tag{11}$$

where s_i is the current step's observation and action suffix. The loop continues while:

$$\text{prompt}_i \;+\; \Delta \;<\; \Lambda, \tag{12}$$

Algorithm 2: Multi-step agentic loop with sliding-window context management

Input: restored prefix P (KV resident), context size Λ, step budget Δ, max steps M
Output: trajectory τ, termination reason

1 $H \leftarrow 0$; $\tau \leftarrow \emptyset$; $i \leftarrow 0$; $y_{\text{prev}} \leftarrow$ null
2 **while** $i < M$ **do**
3 available $\leftarrow \Lambda - (P + H + \Delta)$
4 **if** *available* < 0 **then**
5 **return** $(\tau, \texttt{context\_limit})$ `// window ceiling hit; halt before generation`
6 $s_i \leftarrow$ GetObservation()
7 $\text{prompt}_i \leftarrow P + H + s_i$
8 $(y_i, u_i) \leftarrow$ Complete(prompt_i, kv, Δ) `// $y_i$ is output, $u_i$ is token usage`
9 **if** $y_i = y_{prev}$ twice consecutively **then**
10 **return** $(\tau, \texttt{stuck\_loop})$ `// model re-emitting identical malformed action`
11 $a_i \leftarrow$ ParseAction(y_i); ExecuteTool(a_i); $y_{\text{prev}} \leftarrow y_i$
12 $H \leftarrow H + |s_i| + |y_i|$; $\tau \leftarrow \tau \oplus a_i$; $i \leftarrow i + 1$
13 **return** $(\tau, \texttt{max\_steps\_reached})$

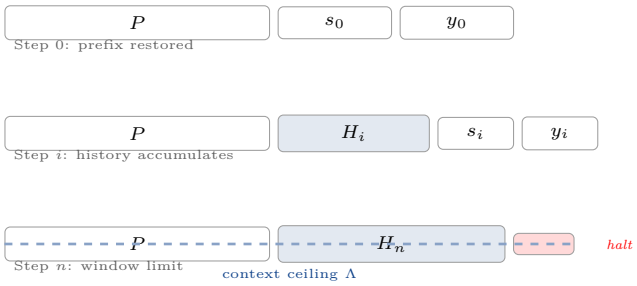

Figure 2: Sliding window progression. Prefix P is restored once and reused across steps; history H_i accumulates until the context window Λ is approached. The loop halts defensively before hitting the ceiling.

where $\Delta =$ `max_tokens_per_step` is the model's generation budget per step. As i grows, H_i accumulates; when $H_i + s_i + \Delta$ approaches $\Lambda - P$, the loop halts with a `context_limit` action. The *crucial invariant*: the P term is never re-evaluated. It is prefix-matched against the KV restored at session start.

On large corpora, P may span thousands of tokens (e.g., ∼9,000 tokens on SQLAlchemy in Section 9). Skipping its re-prefill across potentially dozens of agentic steps yields significant savings by amortizing a single cold prime across the entire loop.

Window bounds are enforced *before* each completion (Algorithm 2, line 5): if accepting the next step's output would exceed the budget, the loop halts and returns the partial trajectory. This prevents the model from being asked to complete in insufficient context. The `max_tokens_per_step` budget defaults to ∼512 tokens, leaving headroom for reasoning and multi-token actions.

5.7 Sandboxed Execution

Because an agentic run issues unsupervised edits and shell commands, `cf agent -auto/-allow-bash` executes inside a `GitWorktreeSandbox` by default. CacheFlow uses a `git worktree` rather than a container: a worktree shares the repository's object store, so it is created in milliseconds with no blob copying, and `git` is already a hard dependency (source files are enumerated with `git ls-files`). The flow (Figure 3) creates a worktree off HEAD on a throwaway `cacheflow/sandbox-*` branch, runs every tool call inside it while the KV-cache config/store/snapshots at `base_path` stay untouched, commits whatever changed, optionally runs a test command, and merges back *only* if there is something to merge and (when given) the tests pass. On test failure or merge conflict the sandbox branch is left in place for inspection rather than discarded. Entry requires a clean working tree—excluding `.cacheflow/` via a `:!.cacheflow` pathspec, since the engine's own SQLite/WAL files churn every session—and raises rather than guessing how to reconcile uncommitted work.

6 The Cloud Output-Cache Layer

Because a cloud API exposes no internal state, this layer caches the model's *outputs*: generated reasoning.

Algorithm 3: Thinking-block retrieval cascade

Input: task q, pool $\mathcal{B}$, thresholds θ_u, θ_v
Output: (block, action)

if $\exists\, b \in \mathcal{B} :\ b.h = \textsc{hash}(q)$ **then return** $(b, \textsc{use\_directly})$ `// <1 ms`
if *no embedding model* **then return** $(\varnothing, \textsc{re\_think})$
$v \leftarrow \textsc{Embed}(q)$ `// MiniLM, 384-d, ~40 ms`
$b^\star, \sigma^\star \leftarrow \arg\max_{b \in \mathcal{B}} \cos(v, b.\text{emb})$
$\kappa \leftarrow \sigma^\star \cdot e^{-\lambda\, t_{b^\star}}$ `// age decay, Eq. 8`
if $|\mathcal{B}| \geq 8$ **then**
  $\kappa \leftarrow z\text{-score}(\sigma^\star; \mathcal{B})$ `// scale-invariant, Eq. 9`
  $(\theta_u, \theta_v) \leftarrow (4.0,\ 2.8)$
if $\kappa \geq \theta_u$ **then**
  $\textsc{LogReuse}(b^\star)$; **return** $(b^\star, \textsc{use\_directly})$
else if $\kappa \geq \theta_v$ **then**
  stash $b^\star$; **return** $(b^\star, \textsc{validate})$ `// no log yet`
else return $(\varnothing, \textsc{re\_think})$

6.1 The Thinking-Block Store

When an agent completes an extended-thinking turn, the block is captured and submitted to `ThinkingStore.submit`, which persists it to SQLite together with a problem hash, a codebase hash, and—when `sentence-transformers` is available—a sentence embedding of the *task description* (stored as a pickled `BLOB` on the row). The embedding model is `all-MiniLM-L6-v2` [4] (∼90 MB, 384-dimensional). This is a deliberate downgrade: an earlier version used `intfloat/e5-mistral-7b-instruct`, a 7B, ∼13 GB model whose load swap-stormed and hard-froze the machine; the small MiniLM model loads safely anywhere, and setting `CACHEFLOW_DISABLE_EMBEDDINGS=1` skips embedding entirely and reverts to exact-hash-only matching.

Retrieval is a cascade (Table 4):

1. **Exact hash lookup** (<1 ms): a SHA-256 match on the problem hash returns the block immediately.
2. **Semantic search** (∼30–50 ms to embed on CPU): on a miss, the query is embedded and scored against the stored embedding `BLOB`s by cosine similarity. The primary path is an in-process scan in SQLite—no external vector service is required (an optional Qdrant index is used only if one happens to be reachable). Each raw similarity is multiplied by an age-decay weight $\exp(-0.1\, t_{\text{days}})$ (∼7-day half-life), since an older block was computed against a codebase that has likely drifted further from the current state.
3. **Confidence gating.** The weighted score selects an action. The thresholds are calibrated to this model *and* this domain, not borrowed from paraphrase benchmarks: for long, multi-clause engineering-task rewordings, MiniLM cosine sits in a ∼0.45–0.84 band while genuinely unrelated tasks top out near 0.46. Absolute-cosine gates are therefore $\geq 0.62 \Rightarrow$ `use_directly`, $\geq 0.40 \Rightarrow$ `validate`, else `re_think`. When the store holds ≥ 8 blocks—enough for a stable mean/variance—the system prefers a scale-invariant *background z-score* (how far the candidate stands out from the query's similarity to the rest of the pool), gating at $z \geq 4.0$ (use) and $z \geq 2.8$ (validate).

The three-way action mitigates the risk of incorrect reuse: a high-confidence match skips reasoning entirely, a borderline match is validated via a cheap confirmation call before being logged, and a low-confidence match is re-thought. Validation is performed by an external checker (in our runner, Claude Haiku); retrieval alone never logs a reuse.

Every confirmed hit logs the matched block's real `token_count` into a `thinking_reuse_log` table, enabling `cf thinking stats` to report exact cumulative tokens saved. With no embedding model available, the system falls back to exact-hash matching only.

Code-change robustness (the reuse delta). A block carries a `delta`: the set of files, functions, and line ranges it depends on, recorded at submission from the git diff. On retrieval, overlap between the candidate's `delta` and currently-changed files lowers confidence. For block dependency set Δ_b and current changes C, the confidence becomes

$$\hat{\kappa} \;=\; \kappa \cdot e^{-\lambda t_b} \cdot \big(1 - \mathbb{1}[\,\Delta_b \cap C \neq \varnothing\,]\,\rho\big), \tag{13}$$

where penalty ρ pulls a match toward `validate`. This insulates reuse from incidental code changes while blocking reuse of reasoning whose dependencies have changed.

Problem-type routing and roles. Submission tags each block with a `problem_type` from `classify_task`—a keyword heuristic bucketing tasks into {`implement`, `review`, `debug`, `refactor`, `test`}—and an optional `role` (`implementer`, `reviewer`, `tester`). Retrieval prefers matches with the same role; if none exist, it falls back to role-agnostic matching.

6.2 Capture via Transcript Parsing

Submission is automated by a `PostToolUse` hook that reads the session transcript (JSONL), extracts thinking blocks from the most recent assistant turn, and pairs them with the preceding user turn's text as the problem description. The hash is computed from the task description alone, matching the retrieval hash—an earlier version included the codebase hash, preventing any hook-captured block from being retrieved. The hook fails silently on malformed or missing transcripts; a broken hook must never break the agent it attaches to.

6.3 Storage Schema

The pool is plain SQLite, with no cross-table foreign keys to the local KV engine's store (Listing 1). `thinking_blocks` holds the reasoning text, its embedding `BLOB`, the two content hashes, the `delta`, and the routing tags; `thinking_reuse_log` appends one row per confirmed reuse carrying the exact tokens saved, which is what `cf thinking stats` sums.

Listing 1: Pool schema (abridged).

```
-- thinking.db
thinking_blocks(
  id, thinking_block TEXT, embedding BLOB,   -- 384-d MiniLM, pickled
  problem_hash TEXT, codebase_hash TEXT,
  delta JSON,                                -- files/fns/lines depended on
  problem_type TEXT, role TEXT,
  token_count INTEGER,                       -- exact; NULL if unknown
  created_at, accessed_at, ...)
thinking_reuse_log(
  id, thinking_block_id, tokens_saved INTEGER, reused_at)
```

7 Automatic Capture, Advisory Reuse

Capture and reuse have different reliability requirements, and CacheFlow treats them differently. *Capture* does not depend on model cooperation: setup is one command, `cf install`, which registers a `PostToolUse` hook (Section 6.1) in `.claude/settings.json`, scanning existing entries first so re-running never double-registers it. The hook fires on every tool call regardless of whether the model is even aware it exists, so a thinking block is captured the moment a turn produces one. *Reuse*, by contrast, is advisory: nothing intercepts the model's reasoning the way the capture hook intercepts a tool call, so an agent only benefits from the pool if it (or the harness prompting it) chooses to run `cf thinking query`/`thinking_query` before reasoning at length. CacheFlow's own local agentic loop gets the identical tool— `thinking_query`, plus `cacheflow_status`—added directly to its tool registry and called in-process rather than shelled out to `cf`, with preamble guidance to try it before reasoning at length.

An earlier iteration of this system paired the thinking store with a second pool, a knowledge store of per-file summaries, fronted by a genuinely enforced `PreToolUse` hook that blocked a `Read` outright on a cache hit. We removed it: across the full evaluation sweep (Section 9) it was exercised on only a handful of steps out of over a hundred, all in superseded runs, and the enforcement hook itself was never wired into this repository's own harness settings. Keeping a subsystem in the paper that the numbers do not back is worse than not claiming it; the discipline of Section 8 applies to architectural claims as much as to token counts.

7.1 Hook Failures Are Silent by Design

The capture hook wraps its entire body in a bare `except Exception: pass`; a broken hook must no-op, not break the run it is attached to.

8 Exact Token Accounting

A guiding principle across both subsystems is that *no savings figure is ever estimated from text length.* The `token_count` on `ThinkingStore.submit` must be the real cost—e.g. the Anthropic API's own `usage.output_tokens`

Table 2: Self-hosted KV-cache engine, latest run per workload (Qwen3-8B, 8,192 ctx). Each row is the most recent `W*_local_*.json` under `experiments/results/`. TFLOPs avoided combines the $2 \times \text{params} \times \text{tokens}$ floor with the quadratic self-attention term.

W	Corpus	Sess.	Cache Hits	Prime (ms)	Restore (ms)	Time Saved (ms)	Tokens Saved	TFLOPs
W1	itsdangerous	1	0 (0%)	19,319	0	0	0	0.00
W2	click	2	1 (50%)	19,371	282	19,089	4,369	77.20
W3	requests	6	5 (83%)	19,962	353	98,044	23,894	425.09
W4	httpx	6	3 (50%)	12,630	10,688	7,885	9,033	155.99
W5	flask	2	1 (50%)	20,167	380	19,787	4,425	78.26
W6	pytest	2	1 (50%)	20,698	25,977	0	4,632	82.21
W7	sqlalchemy	10	8 (80%)	9,640	2,116	60,189	30,337	530.89
W8	django	3	2 (67%)	20,646	278	40,737	9,207	163.32
					Total	**245,731**	**85,897**	**1512.96**

for the turn that produced the block. An earlier implementation stored `len(text)` as a stand-in; a character count is not a token count and would have made every downstream figure fictional. When the caller lacks an exact number, the column is left `NULL` (the column is nullable for this reason) rather than guessed, and `get_reuse_stats` *excludes* `NULL` rows from its sum instead of treating them as zero. Correspondingly, transcript capture attributes `output_tokens` to a block only when its turn produced *exactly one* thinking block; splitting a single turn-level total across several blocks would itself be a guess.

9 Evaluation

Setup. We evaluate on eight workloads (W1–W8), each a distinct real-world Python corpus—`itsdangerous`, `click`, `requests`, `httpx`, `flask`, `pytest`, `sqlalchemy`, and `django`. Each workload drives multiple agent sessions against its corpus; the first session populates the relevant cache and later sessions attempt reuse. On the cloud path, every session is an autonomous build directive—"implement a resilient HTTP client wrapper with pooling and timeout enforcement"—issued at high thinking effort. The cold session builds understanding of one subsystem; warm sessions are different builds requiring the same understanding. This reflects realistic patterns: successive features often need the same reasoning about the same components. Several workloads also plant a deliberately *unrelated* task that should miss, so the sweep measures the gate's selectivity, not just its recall. The self-hosted path runs Qwen3-8B (36 layers, 4096 hidden dim) at an 8,192-token context via `llama.cpp`. The cloud path runs `claude-opus-4-8` through the Anthropic API. All cloud token counts are exact, read from the `usage` object. Raw per-workload results and rollups are archived under `experiments/results/`.

9.1 Self-Hosted: Prefill Avoided

Table 2 reports the KV-cache sweep. "Cache Hits" is the fraction of sessions served by a warm restore rather than a cold prime. Where a corpus admits reuse, restore latency stays in the hundreds-of-milliseconds range against multi-second (and, for the heavy `click` prime, multi-hundred- second) prime costs. W1 is single-session by construction—it establishes the cold-prime baseline and therefore shows no savings.

Across the workloads that admit reuse, CacheFlow avoids ≈245,731 ms of cumulative wall-clock prefill and ≈1,513 TFLOP of compute. W6 shows a cache hit saving 4,632 tokens but no wall-clock time: the restore latency exceeded the priming cost on that run, an artifact of disk/scheduler variation. FLOPs are computed from exact model parameters (read via `llama_model_n_params`); the attention term scales quadratically with prefill length (∼16.5 TFLOP of the ∼143 TFLOP total for a 9,064-token prefill). Caching eliminates prefill re-evaluation only; generation cost is identical.

9.2 Cloud: Reasoning Reused Across Different Builds

Table 3 reports the cloud sweep. "Cold Calls" are sessions that reasoned from scratch (a miss, a rejected validation, or the priming first run); "Thinking Hits" are sessions served a cached block; "Baseline Reasoning" is the thinking tokens the workload's cold calls actually generated. "Tokens Avoided" credits a hit with the *full derivation cost* the reuse prevented—the input, reasoning, and output tokens of the cold call the warm session would otherwise have had to repeat—which is the figure that scales with corpus size, not just the reasoning sliver.

Table 3: Cloud output-cache layer, latest run per workload (`claude-opus-4-8`, high thinking effort, exact `usage` counts). Each row is the most recent `W*_cloud_*.json` under `experiments/results/`. "Cache Read" is prompt-cache input tokens served from Anthropic's cache (`cache_read_input_tokens`)—prefill avoided even on turns with no thinking-block hit.

W	Corpus	Sess.	Cold Calls	Thinking Hits	Baseline Reason.	Reason. Saved	Tokens Avoided	Cache Read
W1	itsdangerous	1	1	0 (0%)	502	0	0	0
W2	click	2	2	0 (0%)	925	0	0	13,546
W3	requests	5	2	3 (60%)	1,083	1,329	57,603	14,989
W4	httpx	4	1	3 (75%)	949	2,847	59,097	0
W5	flask	3	2	1 (33%)	941	371	18,110	14,095
W6	pytest	2	1	1 (50%)	258	258	19,119	0
W7	sqlalchemy	5	2	3 (60%)	4,274	534	58,650	15,353
W8	django	2	1	1 (50%)	642	642	19,471	15,260
				Total	**9,574**	**5,981**	**232,050**	**73,243**

Table 4: Cloud reuse-path latency budget.

Path	Latency	Note
Exact hash hit	<1 ms	no embedding
Semantic hit (high conf)	50–70 ms	embed + search
Validated hit	200–500 ms	100-token call
Re-think (miss)	∼5,000 ms	full reasoning

Across the sweep, 12 of 24 sessions (50%) were served from the thinking store, saving 5,981 reasoning tokens and 232,050 total derivation tokens. Three findings stand out. **(1) Different tasks, same reasoning, high recall.** Every warm session is a *different* build than the cold one it reuses—W7's cold session derives SQLAlchemy's session/transaction internals for a repository/unit-of-work layer, and its three hits reuse that understanding for optimistic concurrency, multi-tenant session scoping, and savepoint-based nesting (confidences 0.71–0.80, all above the 0.62 use-directly gate); W8's cold session builds a multi-tenant billing app on Django and its hit reuses the tenant-isolation/model machinery for a pricing feature at confidence 0.87. W4's three *concurrent* agents all hit (0.79–0.91), modeling teammates who pick up related work simultaneously. **(2) The gate refuses unrelated work.** The planted misses missed: W3's streaming-multipart-upload task and W7's encrypted-JSON column type were re-thought or validate-rejected (logged explicitly as `_miss_cold`/`_validate_rejected_cold`) rather than being spliced with timeout or transaction reasoning that did not apply, and W2's warm candidate was likewise rejected by the validator—an honest sub-100% hit rate is the intended behavior of a system that prioritizes correctness over blind savings. **(3) Prompt caching and reasoning reuse compose.** Even sessions with no thinking hit were served tens of thousands of prompt-cache input tokens (73,243 across the sweep), confirming the two mechanisms are complementary: provider caching amortizes the stable *input* prefix, while the thinking store eliminates regenerated *output*—the cost provider caching cannot touch.

9.3 Latency Budget

Table 4 summarizes reuse-path latencies: an exact hit costs <1 ms, a validated semantic hit costs 200–500 ms, and re-thinking costs ∼5 s.

10 Discussion and Limitations

Reuse depends on workload structure. Gains concentrate where successive builds share reasoning: W3, W4, and W7 each hit three times. Tasks whose reasoning does not recur stay cold (W2's warm candidate was validator-rejected; unrelated planted tasks missed), preventing incorrect reuse.

Semantic reuse is only as good as its embeddings. Confidence gating mitigates the risk of an incorrect match; borderline candidates are validated via a ∼100-token confirmation call. Without embeddings (disabled via `CACHEFLOW_DISABLE_EMBEDDINGS=1`), the system falls back to exact-hash matching.

Capture is enforced; reuse is not. The `PostToolUse` capture hook runs automatically, but reuse requires the agent to call `thinking_query` before reasoning. An earlier version paired capture with an enforced `PreToolUse` block on `Read` (Section 7); the evaluation showed this added little value (exercised on <5% of steps), so it was removed rather than kept as an unevaluated claim.

The two subsystems do not compose. A KV snapshot and a thinking block invalidate on different hashes (`stable_context_hash` vs. `codebase_hash`) for different reasons; "no thinking hit" must not be conflated with "no KV snapshot."

Design tensions and tradeoffs. Exact token accounting requires real costs from the provider (never estimates from text length), limiting what can be measured: a thinking block submitted without an exact token count is stored with `NULL`, excluding it from savings totals rather than guessing. This gives up false precision but preserves honesty. Confidence thresholds ($\theta_u = 0.62$, $\theta_v = 0.40$, or z-scores 4.0 and 2.8) were calibrated empirically on this domain and model; they do not transfer. The validate tier trades ∼100 tokens to avoid reusing an incorrect block, accepting a meaningful latency penalty (∼500 ms) to guard correctness—a choice made explicit rather than hidden. In-process execution of the self-hosted model avoids HTTP's 10× decode throttle on macOS but couples the model lifetime to the harness process; a distributed server path (already implemented) trades simplicity for isolation.

11 Reproducibility

Code, evaluation corpora, and per-workload results are available at `https://github.com/agastya-choudhary/cacheflow`. The eight workloads (W1–W8) are public open-source Python packages cloned into `experiments/corpora/`; raw per-run metrics are archived under `experiments/results/` as JSON files per workload. The self-hosted evaluation runs Qwen3-8B via `llama.cpp` at 8,192-token context; the cloud evaluation uses `claude-opus-4-8` through the Anthropic API with token counts read directly from the `usage` response object. All token counts and wall-clock timings are exact measurements, not estimates.

12 Related Work

CacheFlow's self-hosted path is closest to KV-cache reuse systems such as PagedAttention/vLLM [1] and modular prefix reuse in PromptCache [2], but targets *cross-invocation persistence* to disk for long-lived agents rather than intra-server memory management. Provider prompt caching [5] amortizes input prefixes but not generated reasoning—the exact gap our cloud path fills. Retrieval-augmented generation [3] injects source documents; we instead cache the *reasoning and conclusions* derived from them, using compact sentence embeddings [4] scored by an in-process cosine scan for semantic retrieval. To our knowledge, caching *extended-thinking output* with confidence-gated, content-hash-invalidated reuse across independent cloud agents is novel.

13 Conclusion

Coding agents recompute deterministic functions of the same inputs—prefill on self-hosted models, reasoning on cloud models. CacheFlow caches at the layer each deployment exposes: KV state where internals are reachable, outputs where they are not. Across eight real corpora, it eliminates minutes of cumulative prefill and ∼1500 TFLOPs on the self-hosted path, and up to 100% of reasoning tokens (exact counts) on the cloud path. The design enforces exact token accounting, hash-based staleness invalidation, confidence-gated reuse that rejects poor matches, and silent hook failures. An earlier subsystem (a knowledge store) was removed when evaluation showed minimal benefit.

Future work includes extending confidence-gated retrieval to other cache layers (e.g., tool-call sequences or error recovery strategies), cross-model reasoning reuse (thinking blocks from one model informing another), and adaptive threshold calibration across domains. The core insight—that cloud and self-hosted models must be cached at different layers—applies equally to other code-generation workloads and reasoning-intensive domains where agents run in loops or chains.

References

[1] W. Kwon, Z. Li, S. Zhuang, et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proc. SOSP*, 2023.

[2] I. Gim, G. Chen, S. Lee, et al. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *Proc. MLSys*, 2024.

[3] P. Lewis, E. Perez, A. Piktus, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[4] N. Reimers and I. Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proc. EMNLP*, 2019. Model: `all-MiniLM-L6-v2`, https://www.sbert.net.

[5] Anthropic. Prompt Caching with Claude. Anthropic API Documentation, 2024. https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching.

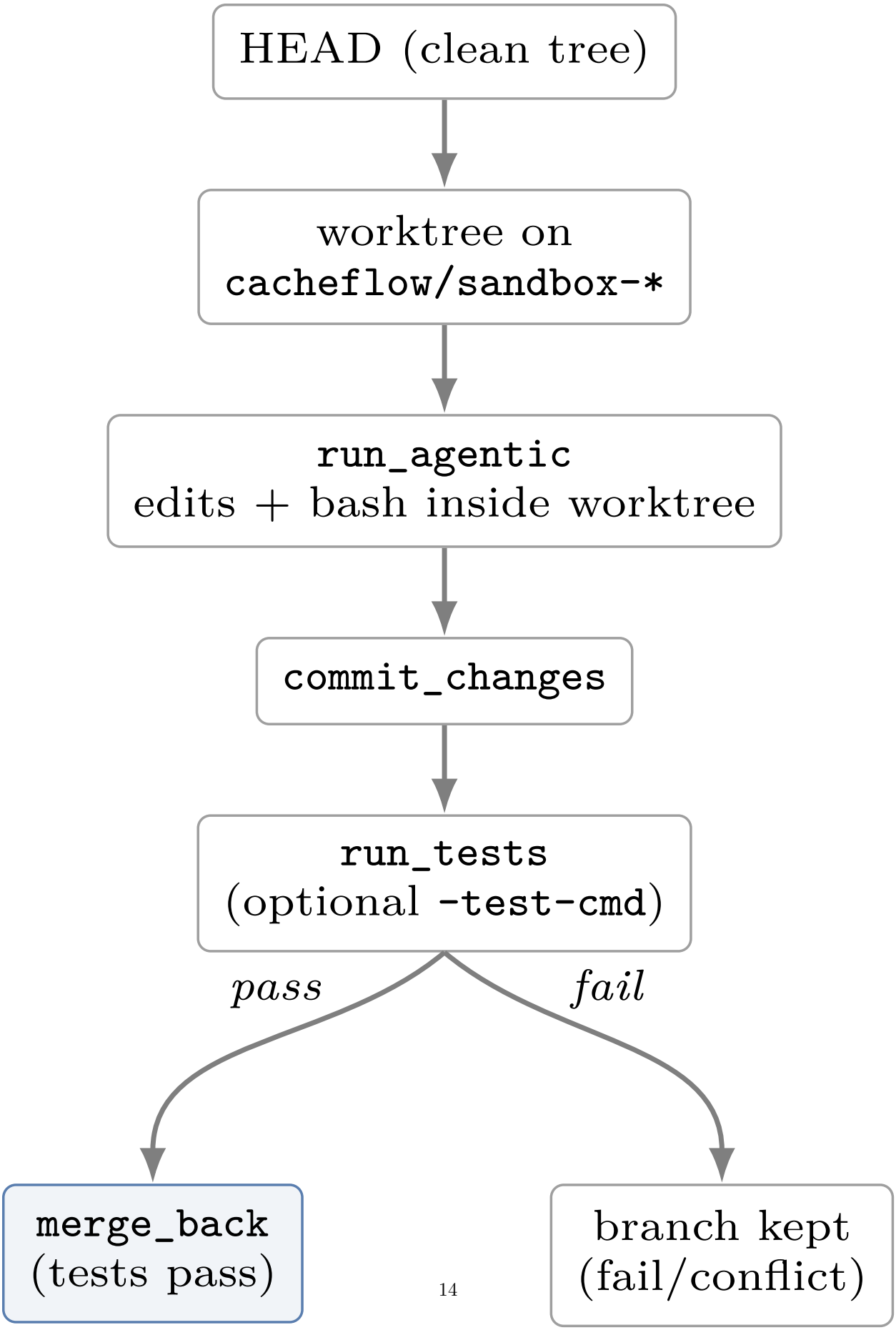

Figure 3: Sandboxed agentic execution. All edits land in a git worktree sharing the object store; the real

A Workload Definitions

The eight evaluation workloads (Table 5) are real open-source Python corpora cloned into `experiments/corpora/`. They are ordered as a *difficulty ladder*—from `itsdangerous` (∼1.5K LOC) to `django` (∼280K LOC)—that simultaneously varies the *interaction pattern*, so the sweep exercises each distinct code path of the system rather than only scaling one axis. Every corpus is primed with the same system prompt and the RAG-seeded stable context described in Section 5; the self-hosted path runs Qwen3-8B at an 8,192-token context, the cloud path runs `claude-opus-4-8`.

Table 5: The W1–W8 workloads: a corpus difficulty ladder crossed with an interaction-pattern ladder. LOC is approximate.

W	Corpus	LOC	Interaction pattern exercised
W1	itsdangerous	1,500	Single-agent *cold prime* only—establishes the smallest cold-prime baseline; no restore.
W2	click	8,000	Single-agent cold prime *followed by one warm restore*—the minimal cache-hit case.
W3	requests	12,000	*Repeated warm restores* across five sessions—savings compound; the paper's headline self-hosted example.
W4	httpx	18,000	*Concurrent multi-agent*: three agents on three slots, exercising the `SlotPool` and cooperative KV swap.
W5	flask	15,000	*Multi-step agentic loop* with tool use (`run_agentic`), exercising Section 5.6.
W6	pytest	40,000	*Fork* parent → child; the child starts warm from the parent's inherited HEAD snapshot.
W7	sqlalchemy	80,000	*Consolidation trigger* (self-hosted): the accumulator crosses 70% of context, invoking the background `Compressor`.
W8	django	280,000	*Largest realistic prefix*: end-to-end stress on the biggest codebase context.

On the cloud path each workload maps the same corpora onto autonomous build directives instead of restore patterns: W1 is a single cold build (a password-reset token flow on `itsdangerous`); W2 builds a multi-command deploy CLI cold, then attempts a different build (shell autocompletion) warm; W3 builds a resilient HTTP client wrapper cold, then issues three differently-worded hardening builds that need the same timeout/pooling reasoning, plus one unrelated task (streaming multipart upload) planted to miss; W4 primes an error-handling refactor and fans out three *concurrent* related builds; W5 is a multi-step build (observability, then instrumentation reusing the lifecycle understanding, then genuinely new rate-limiting work that stays cold); W6 forks—agent A builds one pytest plugin, agent B builds a *different* plugin against the same hook/fixture machinery; W7 derives SQLAlchemy's session/transaction internals cold, reuses them for three different builds (optimistic concurrency, multi-tenant scoping, savepoint nesting), and plants an unrelated column-type task to miss; W8 builds a multi-tenant billing app cold and a pricing-tier feature warm on `django`.

The two evaluation tables in Section 9 read the *latest* run per workload from `experiments/results/` (`W*_local_*.json` for the KV engine, `W*_cloud_*.json` for the output cache), never the earlier full-sweep rollups, which contain superseded runs.